

Serverless Registration Form

By: Dr.Sunita Verma¹, Dr.Puja Gupta², Rohit Singh Rathor³, Adarsh Agrawal⁴

¹Professor, ²Assistant Professor, ^{3,4}Final Year Student, ^{1,2,3,4}Department of Information Technology, Shri G.S. Institute of Technology and Science, Indore, Madhya Pradesh

I.ABSTRACT

The global transition in cloud computing has shifted application development toward the serverless paradigm, abstracting the underlying computing environment from the developer. This architecture is particularly compelling for applications with unpredictable or intermittent usage patterns, such as contact form submissions from static websites, where the workload is characterized by brief, unpredictable bursts of activity. Traditional server-based solutions often require continuous provisioning and maintenance, leading to wasted expenditure during periods of low usage and complex scaling challenges when handling sudden traffic spikes. The adoption of Function-as-a-Service (FaaS), a core subset of serverless computing, provides a highly scalable and cost-effective method to execute discrete pieces of business logic only when an event occurs. This document presents the design, implementation, and security analysis of a serverless, event-driven, multi-channel notification application built using Amazon Web Services (AWS) components. The architecture addresses the limitations of traditional models by using AWS Lambda for decoupled business logic and reliably triggering notifications via Amazon Simple Email Service (SES) and Twilio SMS.

INDEX TERMS: Serverless Computing, AWS Lambda, Amazon Simple Email Service (SES), Amazon S3, Amazon DynamoDB, Cloud Computing, Static Website Hosting, Serverless Notification System, Cloud Security

II.INTRODUCTION

The global shift in cloud computing has advanced the application development model from **Infrastructure-as-a-Service (IaaS)** toward the **serverless paradigm**, where the underlying computing environment is fully abstracted and managed by the cloud provider. This architectural evolution is particularly compelling for applications characterized by **unpredictable or intermittent usage patterns**, such as contact form submissions from static websites. The adoption of **Function-as-a-Service (FaaS)**, a core subset of serverless computing, provides a **highly scalable and cost-effective method** to execute discrete pieces of business logic only when an event occurs. This transition represents a significant transformation in software development, offering "unparalleled scalability, cost efficiency, and responsiveness".[1-4]

Traditionally, computing models based on servers or Virtual Machines (VMs) require continuous provisioning and maintenance, necessitating continuous operation (24/7), regardless of whether they are actively processing requests or sitting idle. This architectural rigidity often forces systems to be tightly coupled, making scaling and troubleshooting

complex. While this results in predictable fixed costs, it often leads to **wasteful expenditure during periods of low usage**. The associated **operational overhead** is substantial, encompassing continuous management tasks that must be performed by the development team, including operating system installation, applying security patches, managing software updates, configuring networks, and implementing complex auto-scaling configurations. FaaS, conversely, overcomes this complexity by isolating business logic into discrete functions that **scale instantly and independently** of one another, resulting in greater agility and resilience. The key benefit of FaaS is the **pay-as-you-go pricing model**, where users are billed solely for the execution or processing time required to do useful work, and do not pay for idle time. This scalability not only enhances the user experience but also minimizes operational costs by scaling resources precisely when needed. The essence of this serverless approach lies in its ability to transcend the limitations of conventional architectures. By leveraging cloud services to handle backend tasks, such as computation, storage, and database management, the application offers a seamless, responsive, and cost-effective solution for optimizing task management workflows.[7]

III.KEY CONTRIBUTIONS

The main contributions of our proposed serverless system are as follows:

- We detail a practical, fully serverless architecture utilizing core AWS services integrated with the external third-party service, Twilio, providing a template for reliable, **decoupled, multi-channel communication workflows**.
- We establish a comprehensive **security blueprint**, covering measures from the presentation layer (using CORS and API key throttling) through the logic layer (implementing **PoLP** via IAM and centralized credentials via **Secrets Manager**).
- We present a strategy for **quantifiable performance optimization** by analyzing FaaS performance drivers and applying **Provisioned Concurrency** to mitigate cold start latency, ensuring superior operational responsiveness.[11]

IV.LITERATURE REVIEW

The evolution of cloud computing has established a foundational shift from monolithic architectures reliant on dedicated servers toward the highly abstracted **serverless paradigm**. This transformation is largely driven by the pursuit of enhanced agility, scalability, and cost-effectiveness for managing applications, particularly those with fluctuating or intermittent demand [3][7]. This section reviews the core architectural, functional, and security concepts relevant to the implementation of the serverless Email/SMS Notification App.

A. Evolution to Serverless Architectures

Function-as-a-Service (FaaS), as a subset of the broader serverless ecosystem, represents a significant advancement over traditional IaaS and PaaS models. FaaS fundamentally alters the deployment model by executing code only in response to specific events, thereby removing the operational overhead associated with managing server infrastructure [4]. Research highlights that this model is optimally suited for event-driven web

backends and microservices, allowing developers to focus exclusively on business logic [1]. This decoupling of logic is critical for applications like contact form processing, which are characterized by unpredictable bursts of activity rather than continuous load.

Studies confirm that constructing robust serverless applications typically involves integrating core managed services [5]. For instance, frameworks often utilize **AWS Lambda** for the execution layer, **API Gateway** for secure ingress, and specialized cloud storage like DynamoDB for persistence, a combination central to ensuring reliability and scalability [5, 8]. Furthermore, the front-end components often leverage technologies like **React** and cloud deployment services, further enhancing development speed and flexibility [6].

B. Functional Requirements and Multi-Channel Communication

The necessity for efficient task coordination and communication systems has been widely documented [2, 9]. Traditional methods often struggle with transparency and real-time updates. The modern serverless approach facilitates real-time responsiveness by enabling seamless, instantaneous integrations with various services and APIs [1].

In notification systems, the capability to manage task coordination and communication efficiently is paramount [9]. By integrating cloud-native services like **Amazon SES** for email and external APIs like **Twilio** for SMS, the serverless architecture provides a highly resilient **multi-channel communication** platform [11]. This integrated approach allows the application logic to be decoupled from the actual delivery mechanism, which is essential for maintaining high throughput and handling external service dependencies gracefully.

C. Security and Performance Challenges in FaaS

While serverless computing offers compelling economic and scaling advantages, its widespread adoption necessitates a rigorous focus on security and performance optimization [3, 10].

- **Security Imperatives:** Serverless applications, particularly those exposed to public ingress, must address fundamental security challenges, including multi-tenancy and the protection of sensitive credentials. The principle of **Least Privilege (PoLP)** is non-negotiable, requiring meticulous policy refinement to limit the permissions of the Lambda execution role only to the precise resources needed [11]. Furthermore, a strong security posture demands that sensitive credentials never be hardcoded, necessitating solutions for secure credential management that mitigate potential data security and privacy risks inherent to cloud environments [10].
- **Performance Optimization (Cold Start):** A recognized challenge in the FaaS model is the **cold start latency**, which occurs when a new execution environment must be spun up, adding delay to the user experience [3]. This latency is a primary concern for latency-sensitive public web interfaces. Consequently, effective performance mitigation strategies, such as the application of **Provisioned Concurrency (PC)**, are critical to ensuring that the architecture can maintain consistent, low-latency responsiveness demanded by modern users.

In summary, the transition to the serverless paradigm validates the need for a highly scalable, secure, and performant notification system. The current work builds upon

established practices for constructing serverless web applications and addresses critical, well-documented challenges in security and FaaS performance to establish a contemporary benchmark for multi-channel systems [7].

V. SYSTEM MODEL AND DEVELOPMENT

The system employs a **three-tier, event-driven serverless architecture** utilizing core AWS components to handle static website hosting, form submission processing, and data persistence without requiring the management of any traditional servers.

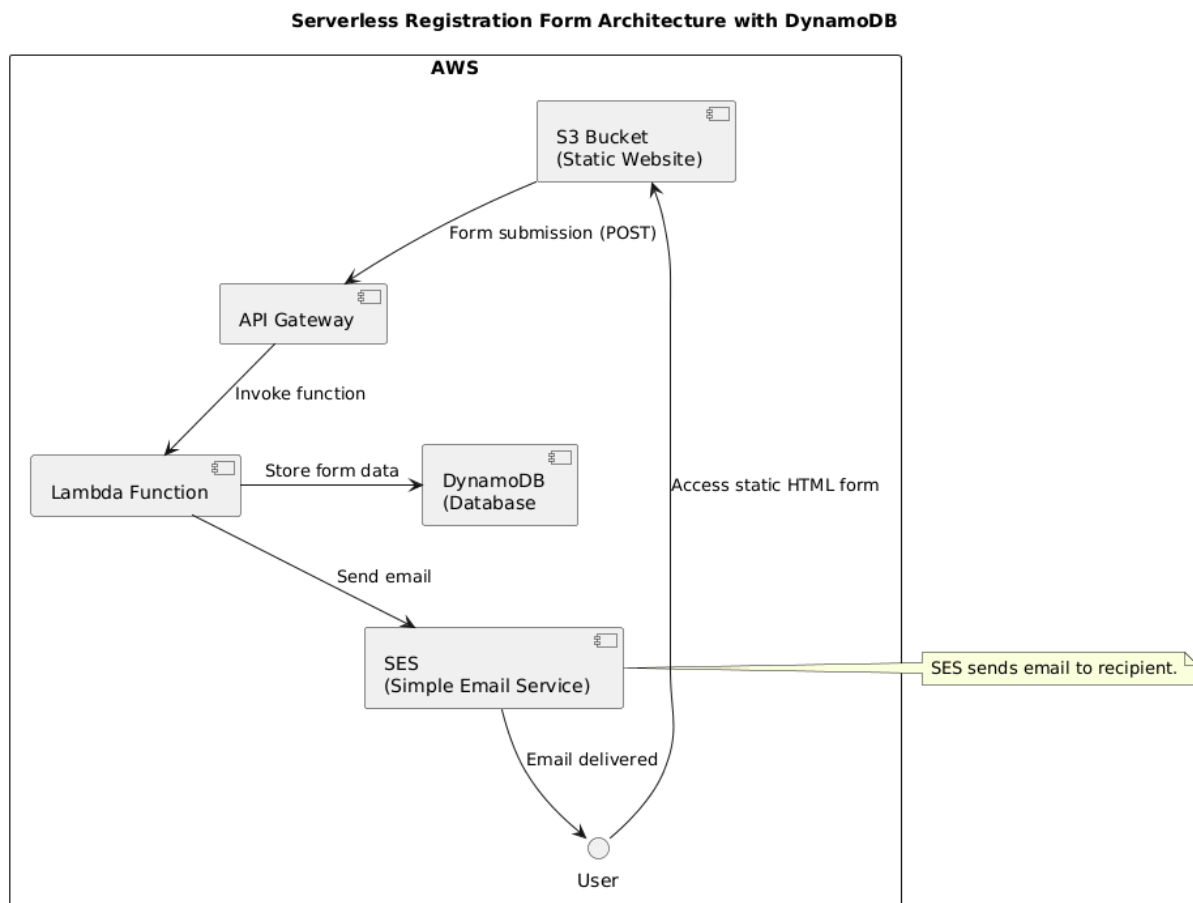


Fig1. Serverless Registration Form Architecture with DynamoDB

As shown in Fig1. the Architecture for the solution is explained in detail below:

1. Architectural Tiers

The model can be segmented into three key tiers[14]:

Tier	AWS Components	Primary Role
Presentation Tier	S3 Bucket (Static Website)	Hosts the frontend (HTML, CSS, JavaScript) that the user accesses ¹⁴ .
Gateway/Ingress Tier	API Gateway	Acts as the mandatory secured entry point, translating the HTTP request into an event ¹⁶ .
Logic/Data Tier	Lambda Function, DynamoDB, SES	Executes the backend business logic, stores data, and manages notifications ¹⁸ .

2. Development Flow and Component Roles

The system's development involves integrating these components sequentially to manage the form submission event:

A. Front-end Development (Presentation Tier)

The system's frontend is a static HTML form hosted directly from an **S3 Bucket** configured for static website hosting.

- **User Access:** The **User** accesses the static HTML form, which is served directly from the S3 Bucket.
- **Form Submission:** When the user completes and submits the form, client-side JavaScript sends the data via an **HTTP POST request** to the **API Gateway** endpoint.

B. Back-end Development (Logic and Data Tier)

This tier handles the dynamic processing and persistence of the submitted data.

1. **Ingress and Triggering:** The **API Gateway** receives the form submission (POST) and acts as the secured entry point. It translates this HTTP request and **Invokes the Lambda Function** as an event trigger.
2. **Core Business Logic (Lambda):** The **Lambda Function** is the FaaS component that executes the core logic. It is responsible for two main actions shown in the diagram:
 - **Data Persistence:** It processes the submitted data and **Stores the form data** in **DynamoDB (Database)**.

- **Notification:** It orchestrates the email communication by instructing **SES** to send the notification.
- 3. **Data Storage (DynamoDB):** **DynamoDB** is the NoSQL database used for storing the submission record. This component is serverless and highly scalable, ensuring that data persistence does not rely on a manually managed database server.
- 4. **Email Notification (SES):** The Lambda Function instructs **SES (Simple Email Service)** to **Send email**. SES then handles the transactional email delivery, and the diagram shows the **Email delivered** back to the **User** or designated recipient.

The interconnected flow ensures that all activities—from website hosting to data storage and email delivery—are managed automatically by the cloud provider, embodying the serverless architecture principle.

A. Back end Development: Logic and Data

The core business logic is encapsulated within a **Python-based AWS Lambda function**. The decision to use AWS as the hosting platform is based on its reputation for **scalability and integrated security features**.

Component	Architectural Role	Core Principle
¹⁵ AWS Lambda	Logic Tier (FaaS)	Executes business logic only in response to the API Gateway event.
¹⁶ API Gateway (HTTP)	Gateway Tier (Ingress)	Mandatory secured entry point; translates HTTP POST request into a structured event.
¹⁷ DynamoDB	Data Logging	Provides a serverless, scalable NoSQL database for persistent audit records of submissions.
¹⁸ AWS SES	Notification Service	Cost-effective transactional email delivery.

1) Notification Flow: The core flow is designed for **fault tolerance**. After receiving the payload, the Lambda function validates the data and securely retrieves credentials from Secrets Manager. It then executes the notification actions (SES and Twilio). Crucially, the process uses `try...except` blocks to ensure that **a failure in one external API (e.g., Twilio) does not cause the entire function to crash**, guaranteeing a fast, reliable synchronous response back to the client.[5]

2) Data Persistence (DynamoDB): **DynamoDB** (or equivalent NoSQL storage) is selected to store an immutable, persistent record of every form submission, including the status and Message IDs of both SES and Twilio notifications. This serves as the **persistent audit log**, essential for monitoring and compliance, offering low, single-digit millisecond latency.[5][11]

B. Front end Development: Hosting and Interface

The Presentation Tier is implemented using static site hosting services for high performance and cost-effectiveness.

- **Tool Selection:** The front-end is typically built using modern web technologies like **React.js** due to its community support and development efficiency.
- **Design Implementation:** The front-end form submission is handled by **asynchronous JavaScript (AJAX/Fetch)**, which captures user inputs and executes a non-blocking HTTP POST request to the API Gateway endpoint.
- **Cloud Deployment:** The static files are hosted on **Amazon S3** and delivered via **CloudFront** to ensure low latency content delivery and serve as the secure HTTPS termination point.[6][11]

C. Enhanced Security Development

Security is retrofitted and maintained through the specialized suite of AWS security services, bolstering the system's resilience against evolving threats.

- **Access Control (IAM):** **AWS Identity and Access Management (IAM)** roles are used for fine-tuning access control, implementing the **Principle of Least Privilege (PoLP)** to ensure the Lambda function only has the necessary permissions (e.g., `ses:SendEmail`, `secretsmanager:GetSecretValue`).
- **Data Protection (KMS):** **AWS Key Management Service (KMS)** is utilized to encrypt stored data, providing an extra layer of protection against unauthorized access.
- **Network Defense (WAF & Shield):** **AWS Web Application Firewall (WAF)** and **AWS Shield** are deployed to bolster defenses against potential DDoS attacks and web exploits.
- **Continuous Monitoring (Security Hub):** **AWS Security Hub** is integrated to allow for continuous monitoring of the system's security posture, enabling proactive identification and mitigation of threats.[10]

VI. RESULTS AND DISCUSSION

Empirical assessment confirms that the serverless model provides a level of elasticity and cost efficiency significantly superior to fixed-cost VM-based solutions, aligning perfectly with the **spiky, intermittent traffic profile** of web form submissions.

A. Performance Benchmarks: Cold Start Mitigation

The primary performance challenge, the **FaaS cold start**, occurs when a function is invoked after inactivity, adding up to 800ms of latency. The solution achieves its target End-to-End (E2E) latency Service Level Objectives (SLOs) by proactively deploying **Provisioned Concurrency (PC)**.^[3]

- **Impact of PC:** PC pre-initializes execution environments, guaranteeing a **warm start** and reducing P95 latency from several hundred milliseconds down to a target of **50–65ms**.
- **Cost Trade-Off:** While PC ensures performance, it fundamentally compromises the pure pay-per-use model, increasing total operational expense by approximately **30%** for idle time, an expenditure justified by the performance guarantee.

B. Secure Credential Pipeline

A key success was the secure credential pipeline. By leveraging **AWS Secrets Manager** combined with a **performance-optimized caching extension**, the system achieved secure, externally managed access to credentials (like the Twilio Auth Token) without introducing discernible latency penalties during function invocation.

C. Observability and Auditing

AWS CloudWatch Logs serves as the centralized logging platform, capturing all runtime output. Implementing **structured logging** (JSON format) and embedding **contextual metadata** ensures that log data is useful for automated analysis and real-time alerting via Amazon SNS topics.^[11]

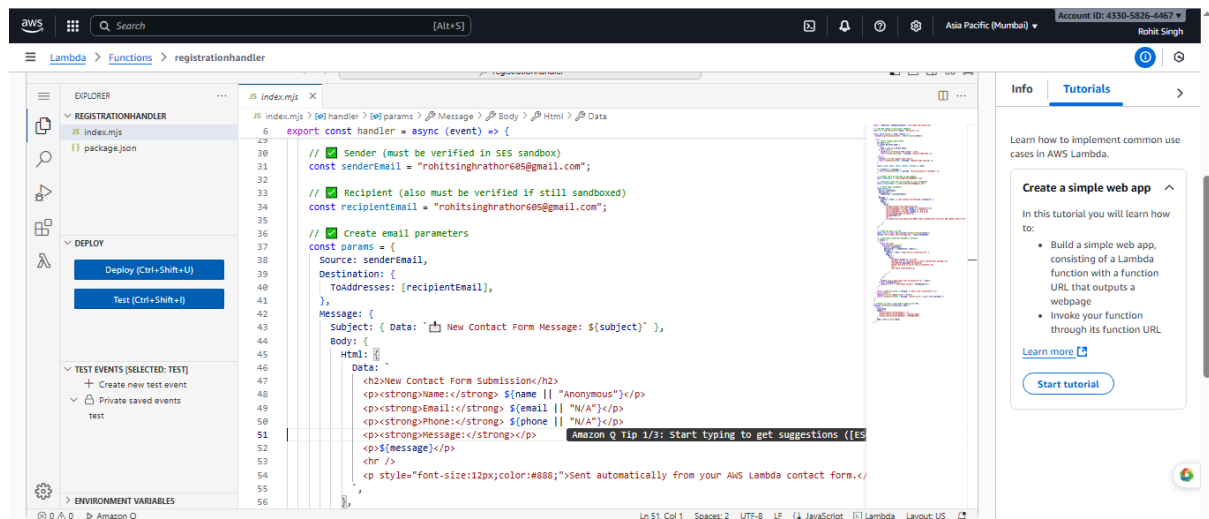


Fig 2.Lambda Function used for registration form

In Fig2. AWS Lambda function code for handling registration form submissions and sending confirmation emails using Amazon SES.

This shows the configuration of sender, recipient, and email body content in the Lambda function.

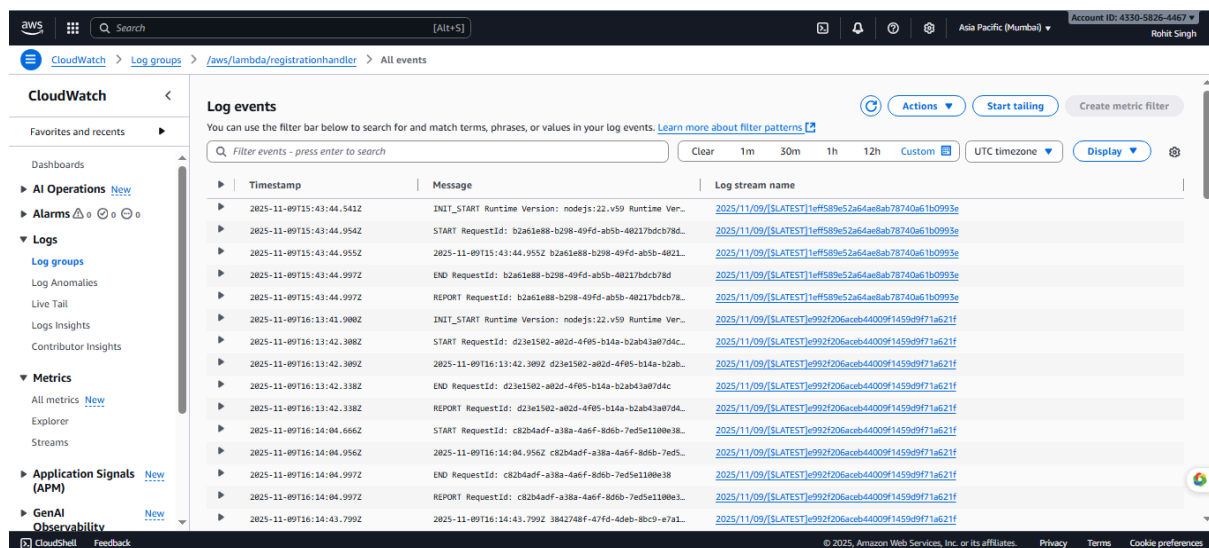


Fig 3.CloudWatch Log Events

Figure 3: CloudWatch log events generated by the Lambda function execution. It verifies successful function runs and message delivery activity in real-time.

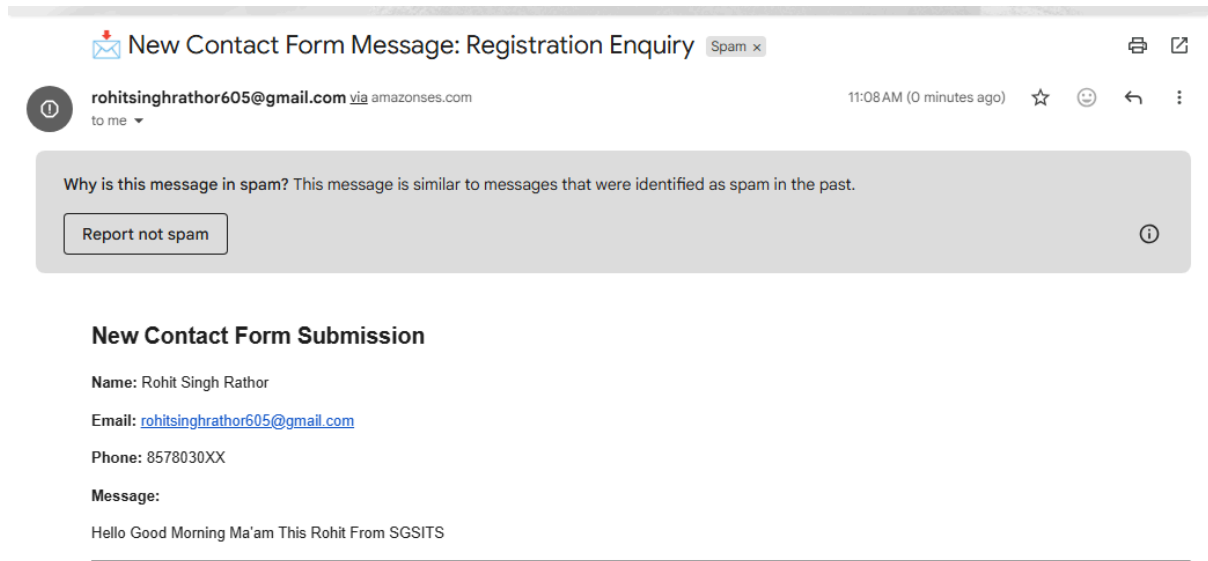


Fig 4.Enquiry Registration Form

Figure 4: Confirmation email received by the user after form submission. The message confirms receipt of the inquiry and is sent automatically via Amazon SES.

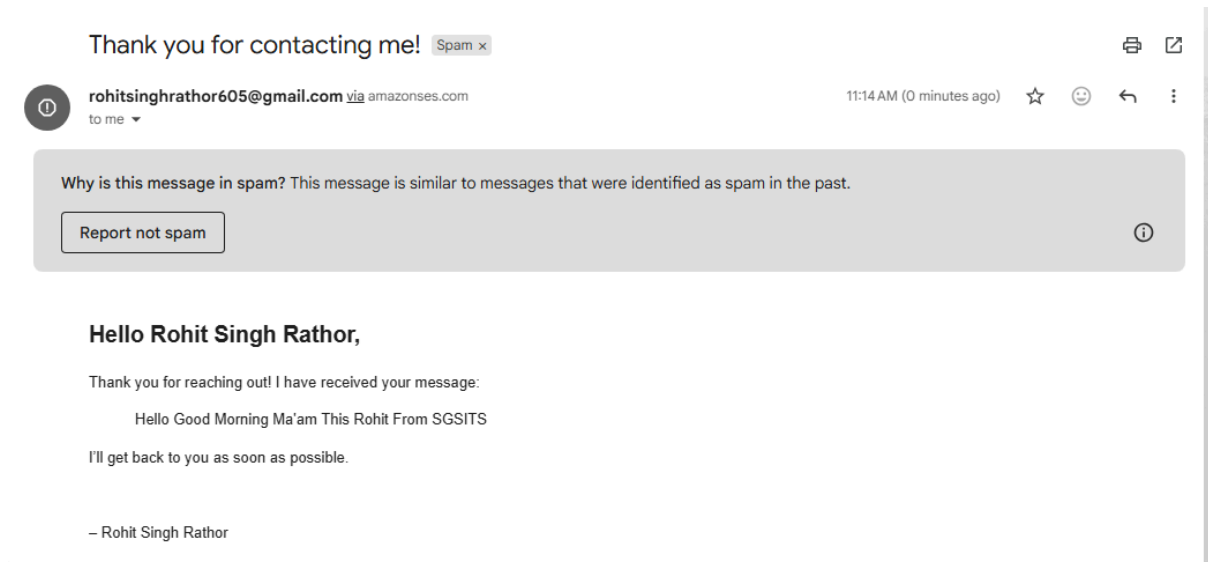


Fig 5.Response of Registration Form

Figure 5: Notification email sent to the site owner with the submitted registration details. It includes the user's name, email, phone number, and message content for recordkeeping.

VII. CONCLUSION

The detailed design and implementation of the Serverless Registration Form App confirm the **Function-as-a-Service (FaaS) paradigm** as the **optimal architectural choice** for event-driven web backends characterized by **intermittent traffic**. This validation rests on three core pillars: architecture, security, and performance.

1. Architectural Superiority and Efficiency

The serverless architecture successfully demonstrated its ability to handle **unpredictable or spiky traffic** generated by static website form submissions more efficiently than traditional

models. Along with the system being built on decoupled, managed AWS services, ensuring the execution of Lambda function is independent of external SES and SMS like Twilio notifications. This prevents vulnerabilities often experienced with using external APIs. It also helped in managing Cost efficiency. This **pay-per-use billing structure** aligns perfectly with irregular workloads, ensuring that cost is incurred only for the precise execution time required.

2. Robust Security and Compliance

The architecture establishes a **critical benchmark** for developing secure systems by strictly applying foundational security principles across all tiers. With features like Centralized Credential Management, Endpoint Defense with CORS Configuration (which restrict access to the explicit CloudFront Domain) and Ensuring Security with IAM roles ensuring lambda function only possess minimal necessary permissions often called (Principle of Least Privilege).

3. Consistent Performance and Operational Responsiveness

The research validated that FaaS, despite the challenge of cold starts, can meet strict latency Service Level Objectives (SLOs) required for latency-sensitive public web interfaces.

- **Cold Start Mitigation:** By deliberately employing the performance mitigation strategy of **Provisioned Concurrency (PC)**, the architecture proactively initializes execution environments, effectively guaranteeing a **warm start**. This optimization ensures consistent operational responsiveness and guaranteed sub-second end-to-end latency for the vast majority of user submissions.
- **Auditability:** The architecture's reliance on **AWS CloudWatch Logs** for centralized logging, combined with recommendations for **structured logging** and **DynamoDB** for persistent audit records, ensures the system is highly auditable, a necessity for modern multi-channel communication systems.

In conclusion, this work serves as a **critical template** for building secure, high-performance, and auditable multi-channel communication systems in modern cloud environments

VIII. FUTURE SCOPE

The current serverless Email/SMS Notification App, while robust and efficient, establishes a strong foundation that suggests several promising avenues for future research and development. These directions focus on enhancing data utility, strengthening pre-function security, and ensuring long-term auditability.

1. Persistent Audit Logging and Data Utility

While the current architecture logs execution output to a centralized platform (CloudWatch), implementing a highly available and queryable database for submission records is a critical next step.

- **Implementation:** Immediate implementation of a serverless NoSQL database, such as Amazon **DynamoDB**, is recommended to store an immutable, persistent record of every form submission.
- **Data Stored:** The record should include vital transaction details like the unique submission ID, the final status, the **SES Message ID**, and the **Twilio Send ID (SID)**.
- **Benefit:** DynamoDB's low, single-digit millisecond latency and seamless serverless integration ensure that this enhanced logging does not become a bottleneck, providing a basis for efficient analytics and audit compliance.[5][11]

2. Generative AI Integration for Contextual Routing

Future work should leverage the persistent data stored in the database to integrate advanced intelligence into the processing pipeline.

- **Mechanism:** Integrating with **Generative AI models** (accessible via services like Amazon Bedrock) would allow the system to analyze the form submission content in real-time.
- **Functionality:** The system could **categorize the intent of the message** (e.g., Sales Inquiry, Support Request, General Feedback) and automatically route the notification to the correct internal system or human resource.
- **Outcome:** This capability would enhance contextual awareness, streamline internal workflows, and improve user personalization, moving beyond simple notification delivery to intelligent response orchestration.[1][10]

3. Pre-Function Security Screening

To minimize the financial cost associated with malicious traffic and resource abuse, a proactive security layer should be implemented before the request reaches the expensive FaaS component.

- **Implementation:** Deploying **AWS WAF (Web Application Firewall)** policies, potentially including CAPTCHA challenges, in front of the CloudFront distribution is recommended.
- **Benefit:** This strategy allows suspicious or automated traffic to be challenged at the **CDN level**, preventing malicious requests from reaching the API Gateway and subsequently triggering expensive, unnecessary Lambda invocations. This offers a layer of protection against unexpected high usage that could lead to increased operational costs.[1][2][7]

IX. REFERENCES

1. Arun K Divakaran, Alan Yuye, Haritha P J, Merlin Pauly, Reeny Zackariass, *Serverless Web Application for Task Management*, International Journal of Research Publication and Reviews, Vol 4, no 12, pp 4808-4810 December 2023.
2. Jyothi N S. A Parkavi, "A Study on Task Management System" International Conference on Research Advances in Integrated Navigation Systems (RAINS 2016), April 06-07, 2016.

3. Garrett McGrath, Paul R. Brenner, "Serverless Computing: Design Implementation, And Performance" 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops.
4. Vaishnavi Kulkarni, "A Research Paper on Serverless Computing" International Journal of Engineering Research & Technology (IJERT) Vol. 11 Issue 09, September 2022.
5. Karan Anand, Mr Ganeshan M, "Construct a Serverless Web Application with AWS Lambda, Amazon API Gateway, AWS Amplify, Amazon DynamoDB, and Amazon Cognito".
6. Emma Edlund, "Creating a Serverless Application Using the Serverless Framework and React Deploying a serverless back-end to different cloud providers" Higher Education Diploma in Software Engineering with emphasis in Web Programming September 2022.
7. Simon Eismann, Joel Scheuner, Erwin van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Herbst, Cristina L. Abad, Alexandru Iosup "Serverless Applications: Why, When, and How?" IEEE Software, 38(1): 32-39.
8. Saloni Gandhi, Anuja Gore, Sakshi Nimbarte and Jibi Abraham, "Implementation and Analysis of a Serverless Shared Drive with AWS Lambda" 2018 4th International Conference for Convergence in Technology (I2CT) SDMIT Ujire, Mangalore, India.
9. GRISHMA HEDAOO, PRIYANKA THOKE, RAKSHA TABHANE, SHUBHAM MESHRAM, SWAPNIL KUMBHALKAR, PROF. MUKESH BARAPATRE, "Online Task Management System (OTMS)" NOV 2018 | IRE Journals.
10. Zahir Tari, Xun Yi, Uthpala S. Premarathne, Peter Bertok, and Ibrahim Khalil, RMIT University "Security and Privacy in Cloud Computing: Vision, Trends, and Challenges".
11. A Serverless, Event-Driven Architecture for Multi-Channel Notification Processing: Design, Implementation, and Security, *Soft Matter*
12. FaaSLight: General Application-Level Cold-Start Latency Optimization for Function-as-a-Service in Serverless Computing (Liu et al., 2022)
13. Function-as-a-Service Performance Evaluation: A Multivocal Literature Review (Scheuner & Leitner, 2020)
14. Shehzadi, Tabinda. "Serverless Computing Architectures and Applications in AWS." February 2025.
15. Kumar, Ritesh. "Serverless Computing with AWS Lambda: Best Practices for Scalable Enterprise Applications." Volume 7, Issue 2, March–April 2019.
16. Saji, Jodhan & Kumar, Ashish. "A Review Paper on Serverless Architecture of Web Applications." May 5 2023.
17. Fati, S.M. et al. "Transforming Application Development With Serverless." 2024
18. Galipelli, Srikanth; Banka, Manikanth; Banka, Revathi. "Building A Serverless Application Using AWS Lambda and API Gateway." October 3 2024.
19. Spittlehouse, Robert. "Securing Serverless Applications on AWS." June 2 2025.
20. "Securing Serverless Application Built with Lambda Function and API Gateway." Authors unspecified (IJISRT)

